Benha University

Shoubra Faculty of Engineering

Communication and Computer Program

Saknny: A Web-Based Platform for Student Accommodation and Relocation

Project Submitted in Fulfillment of the CCE580 Course requirements.

By Students:

Mohamed Ibrahem Ahmed	231903608
Ziad Ayman Sayed	231903634
Adham Mohamed Ayad	231903571
Karen Refaat Wadea	231903760
Zeinab Mohamed Ahmed	231903641
Jana Bakry Moussa	231903639

Supervised By:

Dr. Walaa Gouda

Acknowledgment

We would like to express our sincere gratitude to everyone who contributed to the completion of the Sakkny project. First and foremost, we thank our course instructor for their guidance, valuable feedback, and continuous support throughout the project development process. Their insights helped shape both the technical and analytical aspects of this work.

We also extend our appreciation to our colleagues and teammates for their collaboration, constructive discussions, and shared commitment to delivering a solid and well-structured system. The exchange of ideas and teamwork played a crucial role in overcoming technical challenges and refining the final solution.

Finally, we would like to thank our families and friends for their encouragement and support, which motivated us to stay focused and dedicated throughout the duration of this project.

Abstract

Sakkny is a database-driven web application designed to manage and streamline the process of student housing discovery and reservation. The system is developed using a structured software engineering approach, with a focus on data consistency, scalability, and efficient querying. Core system entities include users, property listings, accommodations, bookings, and reviews, all of which exhibit well-defined relationships.

Given the highly structured nature of the data and the need for strong transactional integrity, the system adopts a relational database model. This choice enables enforcement of constraints, referential integrity, and normalization to reduce data redundancy. The database schema is designed using an Entity–Relationship model and is implemented using SQL-based data definition and manipulation techniques.

The backend architecture exposes application logic through APIs responsible for authentication, authorization, property management, and booking operations. On the frontend, users interact with the system through dynamic interfaces that support advanced filtering, searching, and role-based access control. The system is designed to support future extensibility, such as analytics and recommendation features, without requiring fundamental changes to the underlying data model.

This project demonstrates the application of database design principles, system modeling, and full-stack development concepts to solve a real-world problem in student accommodation management.

Contents

Table of Figures

Table of Lists

Chapter 1:

Introduction

1.1 Problem Statement

University students, particularly those relocating from other cities or countries, face significant challenges in securing suitable accommodation. The current process is fragmented, time-consuming, and often lacks transparency. Students typically depend on unverified social media posts, third-party agents, or informal word-of-mouth networks. This situation leads to several issues:

- Considerable Stress and Anxiety: The pressure to find a safe, affordable, and conveniently located place to live -often in an unfamiliar city-can be overwhelming. The uncertainty around availability, quality, and legitimacy of accommodation options increases students' mental and emotional burden.
- Financial Burdens: In the absence of a trusted platform, students are vulnerable to scams, exaggerated rental prices, high broker fees, and misleading property descriptions. These risks can result in unexpected financial losses or commitments to properties that do not match their advertised quality.
- Wasted Time and Effort: The search for accommodation is inefficient and repetitive. Students must sift through numerous platforms, contact multiple landlords individually, schedule physical visits, and verify property details on their own. This process diverts time and attention away from academic and personal responsibilities.
- Negative Academic and Social Impact: Poor or unstable housing arrangements can directly affect students' concentration, academic performance, physical well-being, and social integration. Lack of suitable housing may also discourage potential students from enrolling in certain universities.

At the same time, landlords struggle to reach the right student audience efficiently and to manage their properties and communications in an organized way.

There is a clear need for a centralized, trustworthy, and efficient platform that simplifies the accommodation search process and seamlessly connects students, landlords, and universities.

1.2 Suggested Solution

To address the above challenges, this project proposes Saknny, a comprehensive web application designed to act as the primary bridge between students, landlords, and universities. Saknny aims to streamline the entire accommodation search, verification, and allocation process by providing secure, user-friendly tools tailored to each stakeholder.

1.2.1 Core System Capabilities

1. Unified Profiles for All Users
    - Student Profiles: Students can create detailed profiles that include their personal information, housing preferences, university affiliation, and academic details (e.g., year of study). This enables personalized property recommendations and better matching between students and landlords.
    - Landlord Profiles: Landlords can manage their properties through dedicated profiles, upload property information, images, and virtual tours, and view student requests and booking history.
    - University Portal: Universities have a dedicated portal to view nearby verified listings, validate student enrollment, and share official announcements related to student accommodation.

2. Verified and Detailed Listings

All property listings on Saknny will undergo a verification process to improve authenticity and safety. Listings will include:

- Comprehensive descriptions (location, size, facilities, rules, rent, and payment terms).
- High-quality images and, where available.
- Reviews and ratings from previous student tenants.
- Verification and rich detail help students make informed decisions while increasing landlords' credibility.

3. Secure Communication and Booking

Saknny will provide an integrated messaging and notification Channel that allows:

- communication between students and landlords within the platform.
- Coordination of viewing appointments, clarification of terms, and negotiation of details.
- A streamlined booking workflow that can support reservation requests, approval/rejection, and initial payment confirmation (depending on integration and institutional policies).

4. Administrative and Analytical Tools

- For landlords, tools to track occupancy, manage multiple properties, and view booking statistics.
- For universities, analytics dashboards to monitor housing availability, and student satisfaction indicators, thereby supporting data-driven decision-making.

1.2.2 Business Benefits

Saknny delivers value to all participating groups:

Students

- Reduced Stress: A centralized, secure, and efficient search process.
- Increased Safety: Access to verified landlords and properties reduces the risk of fraud.
- Cost & Time Savings: Fewer unnecessary site visits, less dependence on brokers, and easier comparison of options.

Landlords

- Targeted Audience: Direct access to a large, reliable pool of potential student tenants.
- Higher Occupancy Rates: Faster and more efficient filling of vacant units through better visibility.
- Streamlined Management: Integrated tools for managing listings, messages, and bookings.
- Increased Trust: Association with universities and verified students enhances their reputation.

Universities

- Enhanced Student Welfare: Supporting students in finding suitable housing improves satisfaction and retention.
- Improved Reputation: Demonstrates institutional commitment to student life beyond the classroom.

1.3 Project Scope

The scope of the Saknny project defines the boundaries of what will be designed, developed, and evaluated within this graduation project.

1.3.1 In-Scope Features

- Design and implementation of a responsive web application accessible via modern browsers.
- User registration, authentication, and role-based access control (students, landlords, and university administrators).
- Creation and management of:
  - Student profiles (preferences, saved properties, booking history).
  - Landlord profiles and property listings (details, photos, availability status).
  - University portal with basic administrative tools and access to reporting dashboards.
- Search and filtering functionalities for students (location, price range, property type, distance from university, etc.).
- messaging channel for communication between students and landlords.
- Booking request and approval workflow.
- Basic review and rating system for properties and landlords.
- Administrative interface for managing users, listings, and reports.
- Documentation of system requirements, design, implementation, testing, and deployment procedures.

1.3.2 Out-of-Scope Elements

To keep the project achievable within academic and time constraints, the following aspects are considered out of scope:

- Native mobile applications (Android/iOS) – only a mobile-friendly web interface will be provided.
- Full online payment gateway integration: payments, if any, will be represented as simulated transactions or status fields.
- Legal document automation (e.g., generation and e-signature of binding rental contracts).
- Integration with external real-estate APIs or government verification systems.
- Advanced AI-based recommendation engines (beyond simple preference-based search and filtering).

1.4 Project Aim and Objectives

1.4.1 Aim

The primary aim of the Saknny project is to design and develop a secure, user-friendly web application that centralizes and optimizes the process of finding and managing student accommodation, thereby improving transparency, safety, and efficiency for students, landlords, and universities.

1.4.2 Specific Objectives

To achieve this aim, the project pursues the following measurable objectives:

1. Requirements Analysis:

Gather and document functional and non-functional requirements from representative students, landlords, and university staff.

2. System Design:

Produce a complete system design, including use-case diagrams, database schema, and user interface prototypes that reflect stakeholders' needs.

3. Implementation:

Develop the Saknny web application using suitable web technologies (e.g., HTML, CSS, JavaScript, and a server-side framework) with a secure database backend.

4. Verification and Testing:

Conduct unit, integration, and user acceptance testing to ensure correctness, usability, performance, and security.

5. Evaluation:

Evaluate the system with a sample of target users, assessing usability, satisfaction, and effectiveness in simplifying the accommodation process.

6. Documentation:

Produce comprehensive technical and user documentation, including installation guidelines, user manuals, and project reports.

1.5 Methodology

The development of Saknny follows a structured methodology to ensure that the final system is reliable, maintainable, and aligned with stakeholders' needs. This section briefly reviews common software development approaches and then explains the methodology selected for this project.

1.5.1 Software Development Methods

Traditional software development methods include:

- Waterfall Model
  A linear, sequential approach where each phase (requirements, design, implementation, testing, deployment, and maintenance) must be completed before the next begins. It is easy to manage and document but inflexible when requirements change.

- Spiral Model:
  Combines elements of design and prototyping in stages, emphasizing risk analysis. It is suitable for large, high-risk projects but can be complex to manage.

- V-Model (Verification and Validation):
  An extension of the Waterfall model that associates each development stage with a corresponding testing phase. It improves quality assurance but still has limited adaptability to changing requirements.

While these models provide clear structure, they are less suitable for projects where requirements may evolve during development, or where continuous feedback from users is important.

1.5.2 Agile Software Development Methodologies

Agile methodologies address the limitations of traditional models by emphasizing:

- Iterative and incremental development.
- Close collaboration with stakeholders.
- Frequent delivery of working software.
- Responsiveness to changing requirements.

Common Agile frameworks include:

- Scrum: Organizes work into short iterations called sprints, typically 1–4 weeks long, with defined roles (Product Owner, Scrum Master, Development Team) and regular meetings (daily stand-ups, sprint planning, sprint review, and retrospective).
- Kanban: Focuses on visualizing work and managing workflow through boards and cards, emphasizing continuous delivery rather than fixed-length iterations.
- Extreme Programming (XP): Emphasizes engineering practices such as pair programming, test-driven development, and continuous integration.

For a student project such as Saknny, Agile -particularly Scrum -is appropriate because it supports continuous feedback, progressive refinement of features, and manageable increments of work.

1.5.3 Project Methodology

The Saknny project adopts an Agile-inspired iterative methodology, following the principles of Scrum but adapted to academic constraints.

Key characteristics include:

1. Iteration-Based Development:

   The project is divided into several short iterations, each focusing on a subset of features (e.g., user registration, property listings, messaging). At the end of each iteration, a working prototype is produced and reviewed.

2. Backlog and Prioritization:

   All requirements are captured as user stories in a product backlog. Features that deliver the highest values such as secure registration and property search are prioritized early.

3. Continuous Feedback:

   Prototype demonstrations to classmates, supervisors, and potential users are used to gather feedback, which is then incorporated into subsequent iterations.

4. Incremental Documentation:
   Requirements specifications, design documents, and test plans are updated progressively rather than only at the end of the project.
5. Quality Assurance:

   Testing is integrated into each iteration, including unit tests for core functions, integration tests for workflows, and usability testing for user interfaces.

This methodology balances the need for formal academic documentation with the flexibility required to adapt the system design based on continuous learning and feedback.

1.6 Project Milestones

The major milestones of the Saknny project are as follows:

Table 1:Project Milestone

M1- Project Initiation and Proposal Approval	Define the problem, propose the solution, and obtain approval from the supervisor/committee.
M2- Requirements Gathering and Analysis	• Conduct interviews or surveys with students, landlords, and university staff. • Document functional and non-functional requirements and initial use cases.
M3 – System Design	• Design system architecture, database schema, and application workflows. • Create user interface wireframes and prototypes.
M4 – Core Module Implementation	• Develop user registration/authentication, role management, and profile modules. • Implement property listing and search functionalities.
M5 – Communication and Booking Modules	• Implement messaging Channel, booking workflow, and basic review/rating features.
M6 – Integration and System Testing	• Integrate all modules, conduct comprehensive testing, and resolve defects.
M7 – Evaluation and Refinement	• Gather feedback from test users, refine features and user interfaces, and optimize performance.
M8 – Final Deployment and Documentation	• Deploy the system to a staging or demo environment. • Complete the final project report, technical documentation, and user manual. • Prepare and deliver the final presentation and demonstration.

1.7 Project Plan

The project plan outlines how activities are scheduled and managed throughout the project lifecycle. Due to academic calendar constraints, the plan is organized into phases rather than precise dates.

Table 2:Project Phases

Phase	Description	Main Activities	Key Output
Phase 1	Initiation & Planning	Problem definition, literature review, proposal writing, methodology selection	Approved project proposal
Phase 2	Requirements Analysis	Stakeholder interviews, requirement specification, use-case modeling	Requirements specification document
Phase 3	System Design	Architecture design, database design, UI prototyping	Design documents and UI mockups
Phase 4	Implementation I	Development of authentication, profiles, and property listing modules	Working core system prototype
Phase 5	Implementation II	Development of search, messaging, booking, and admin features	Feature-complete application
Phase 6	Testing & Evaluation	Unit, integration, system, and usability testing; bug fixing	Tested and refined system
Phase 7	Deployment & Documentation	Deployment to demo environment; preparation of reports, manuals, and presentation	Final system and documentation

Progress will be monitored using simple project management tools (e.g., task boards, checklists) to ensure tasks remain on schedule. Regular meetings with the supervisor will be used to review progress and adjust the plan when necessary.

1.8 Summary

This chapter introduced the Saknny project, a web-based platform designed to address the challenges students, landlords, and universities face in managing student accommodation. The chapter outlined the problem context, presented the proposed solution and its key capabilities, and defined the project scope, aim, and objectives. It also explained the chosen Agile-oriented methodology, listed major project milestones, and summarized the overall project plan.

Subsequent chapters will provide a detailed literature review, system analysis and design, implementation details, testing outcomes, and final evaluation of the Saknny application.

Chapter 2:

Literature Review

2.1 Technical Background and Foundations

Modern student accommodation platforms rely on a set of technical foundations that enable them to serve large numbers of users efficiently while maintaining reliability, security, and ease of use. These platforms are typically web-based systems designed to support multiple stakeholders, manage diverse data types, and scale as demand increases. Understanding these technical foundations is essential to rationalize the approaches and systems discussed in the following sections.

2.1.1 Web-Based Platform Architecture

Most contemporary accommodation platforms are built using a web-based client–server architecture. In this model, users interact with the system through a web interface, while core application logic and data processing are handled on centralized servers. This separation allows systems to be accessed from different devices and locations without requiring specialized software installations.

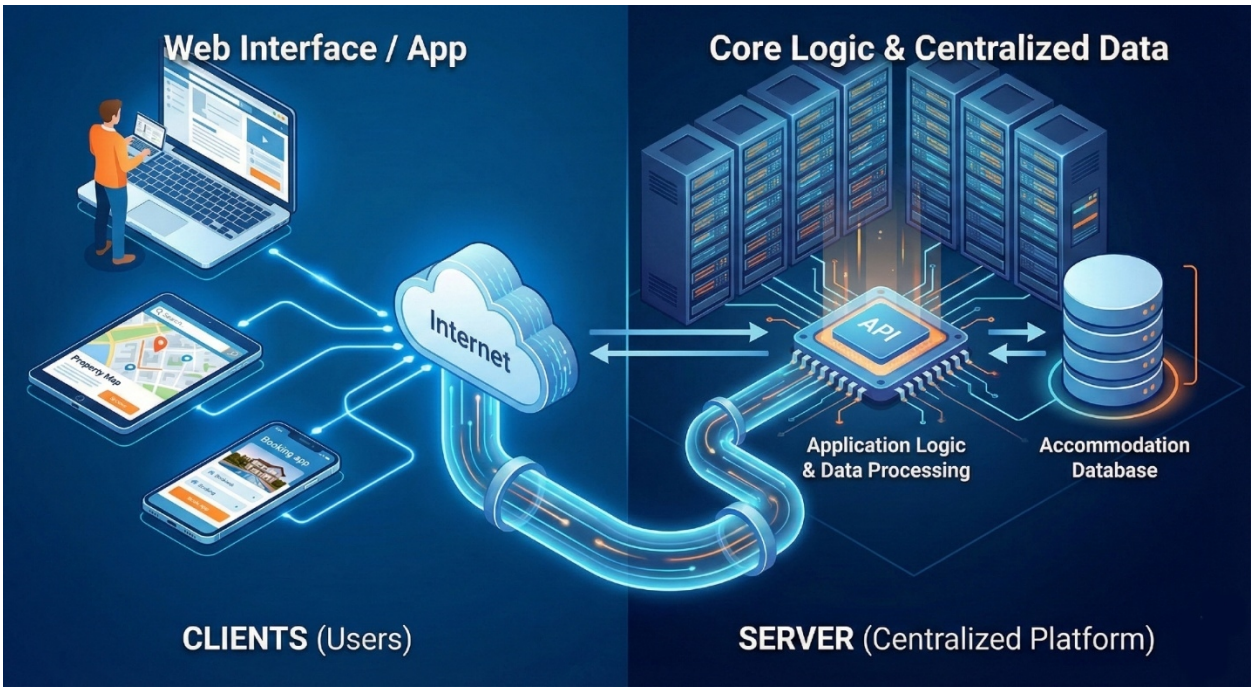

Figure 1:Client-Server Architecture

Web platforms commonly follow a multi-tier architectural structure, where presentation, business logic, and data management are logically separated. Such separation improves system maintainability and allows individual components to be updated or extended without affecting the entire platform. This architectural approach is particularly suitable for applications that evolve over time and require frequent feature enhancements.

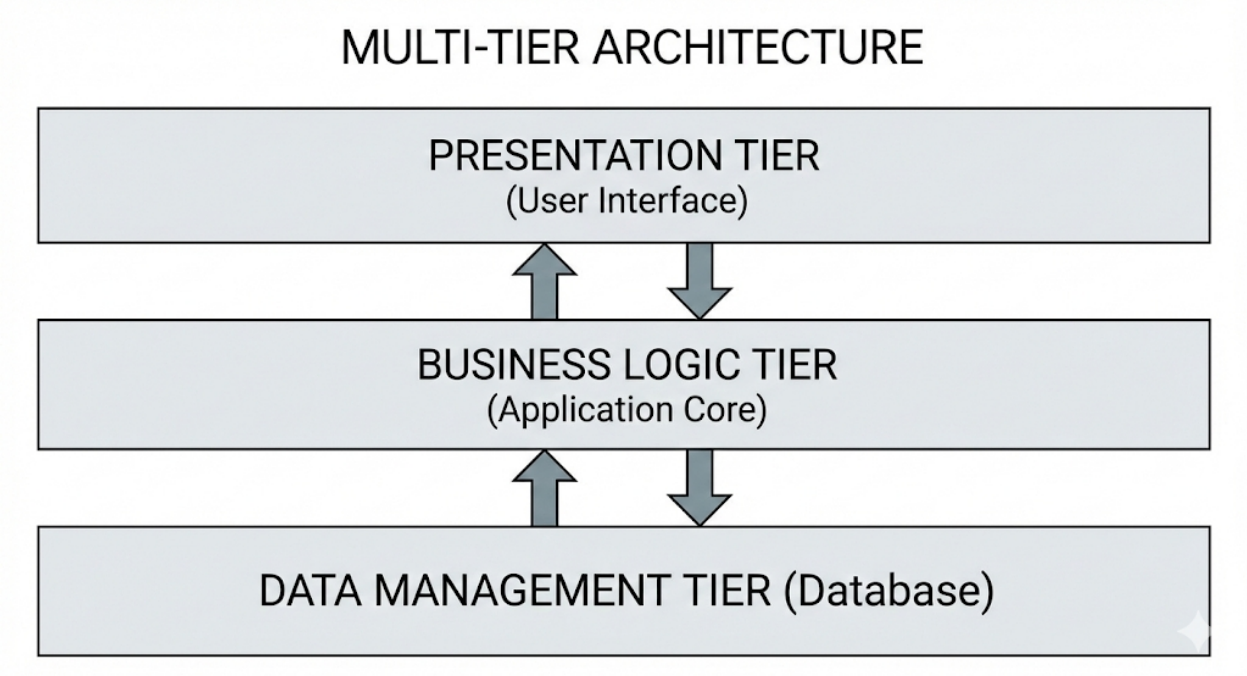

Figure 2:Multi-Tier Architecture

2.1.2 Multi-Role User Systems

Accommodation platforms typically serve multiple user groups with distinct needs and responsibilities. In the context of student housing systems, these users often include students searching for accommodation, landlords managing property listings, and MIS admins overseeing housing-related activities. Supporting multiple user roles requires systems to manage access and functionality based on predefined permissions.

Role-based access models are commonly used to ensure that each user interacts with the system in a controlled and appropriate manner. This approach enhances system security and usability by preventing unauthorized access to sensitive information while providing each user group with tools relevant to their role.

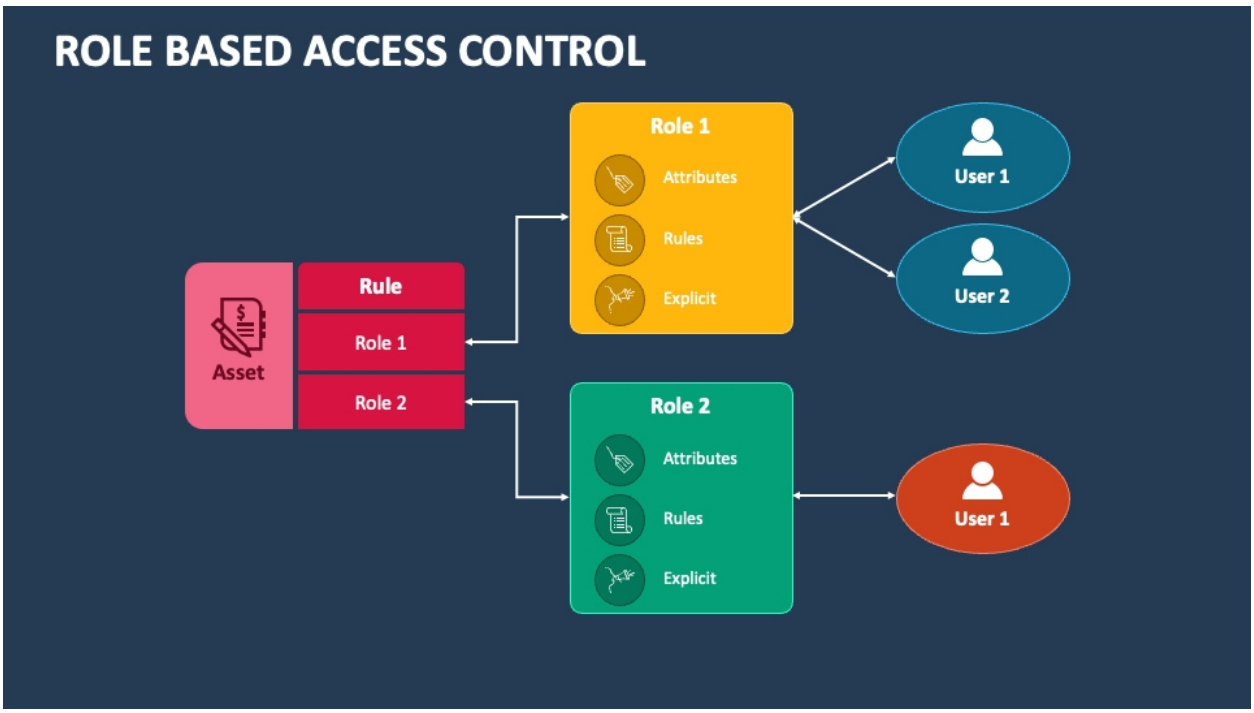

Figure 3:RBAC

2.1.3 Scalability in Web Platforms

Scalability is a key requirement for modern web platforms and is commonly classified into vertical and horizontal scaling. Vertical scalability involves enhancing the capacity of existing system components to support increased demand, while horizontal scalability focuses on expanding the system by adding additional components to distribute workload.

In multi-tenant systems that serve multiple organizations, these scalability approaches are often interpreted differently. Vertical scalability is typically associated with growth in user activity within a single organization, whereas horizontal scalability enables the platform to support additional organizations without significant architectural changes. Such scalability considerations are essential for platforms designed to grow across institutions and regions.

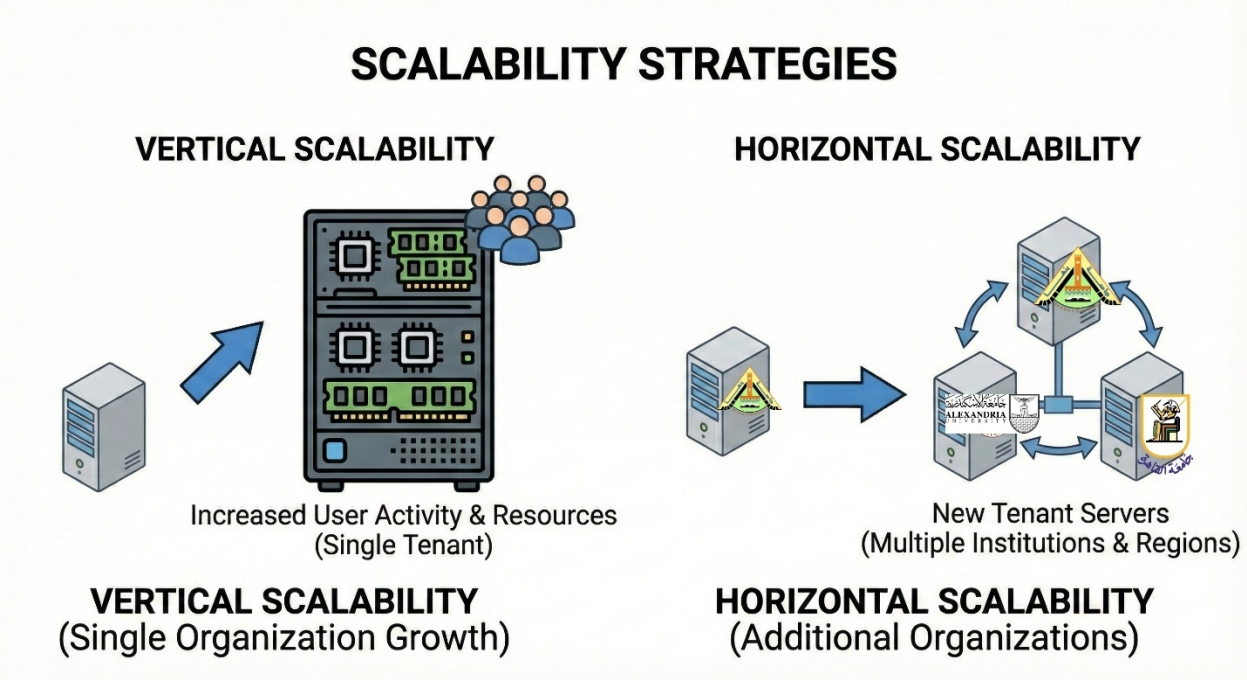

Figure 4:Horizontal vs Vertical Scalability

2.1.4 Data Management and System Consistency

Data management in web-based accommodation platforms typically involves handling both structured and semi-structured data. Relational data models are commonly used to manage highly structured information, such as user accounts, institutional data, and transactional records, where data consistency and integrity are critical. These models enforce predefined schemas, which helps maintain reliable relationships between entities.

In contrast, NoSQL data models are often employed to manage more flexible or rapidly evolving data, such as user-generated content, multimedia metadata, or activity logs. NoSQL systems provide greater schema flexibility and scalability, making them suitable for applications that experience varying data formats and growth patterns.

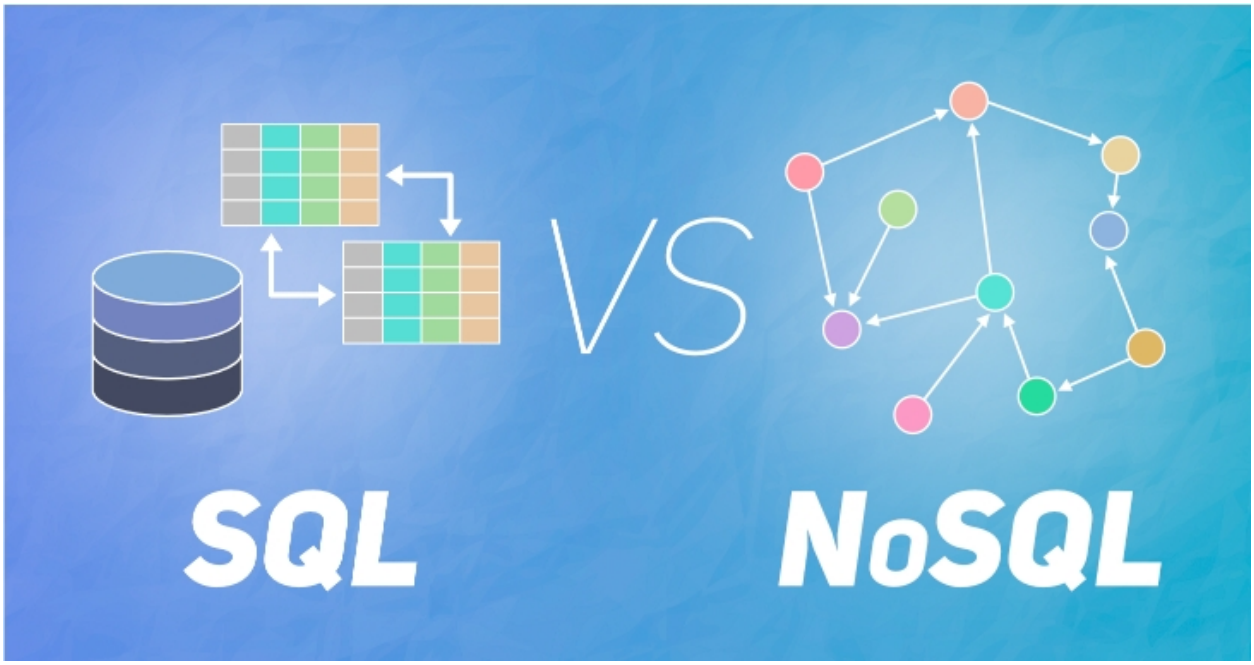

Figure 5:SQL and NoSQL models

2.1.5 Security, Trust, and Verification Foundations

Security and trust are fundamental aspects of online housing platforms, as users rely on these systems when making important accommodation decisions. Basic security principles such as authentication and authorization are commonly used to protect user accounts and restrict access to system resources.

In addition to technical security measures, trust-building mechanisms play a significant role in housing platforms. Verification processes, identity validation, and controlled communication channels are often used to reduce fraudulent activity and improve transparency. These foundational concepts support safer interactions between users and contribute to a more reliable and trustworthy platform environment.

2.2 Related works and Existing approaches

In this section we present an overview of the main approaches in the industry of booking systems and related challenges. Each approach is described in terms of its mechanism and how to manage prior studies if it exists, Strength and Limitations.

2.2.1 Centralized Accommodations

Overview

Centralized housing platforms gather all available rentals into one place. This stops information from being spread across different sites, making it easier for people to find what they need in one place.

How it works

Landlords list their properties on the platform, and the system organizes them into a single database. Instead of checking multiple websites, users can just search, filter, and compare everything through one simple screen.

Prior Studies

Earlier studies show that these platforms help people find housing with less effort because they bring order to a scattered market.

Strengths

- Reduces information dispersion across different channels
- Simplifies the search and comparison process
- Improves visibility of available accommodation options

Limitations

- Just because it's all in one place doesn't mean the info is always 100% accurate.
- The quality of listings can vary a lot
- The platform needs constant checking and quality assurance to make sure the ads stay reliable.

2.2.2 Listing-Based Marketplace

Overview

This approach functions as a peer-to-peer (P2P) marketplace where property owners list their assets and prospective users can review them. Essentially, the platform serves as an intermediary, facilitating the exchange of information.

How it works

Property owners create posts with details like price and location. Users then filter through these ads and reach out to the owners directly, usually off platform to handle the final details and payments.

Prior studies

Studies often point to this model as the "standard" for digital markets because it scales so easily. However, researchers also highlight that trust and safety remain the biggest hurdles for these types of platforms.

Strengths

- User-friendly: The system is straightforward and easy for everyone to use.
- Low Barriers: It’s very easy for new owners or tenants to sign up and get started immediately.

Limitations

- Lack of Control: The platform doesn't have much oversight once the deal moves forward.
- Offline Communication: Since messaging often happens elsewhere, it’s harder to track the communication outside the platform
- Trust Issues:, verifying the quality of a listing can be a challenge additional to building a repetition in the market.

2.2.3Booking-Centric and Transaction-Oriented

Overview

This approach is all about end-to-end management. which handles the entire process from checking dates to the final booking and payment all in one place. It creates a formal, structured way for guests and owners to deal with each other.

How it works

Everything happens in the app. A user finds a place, checks the live calendar, and hits "book." The system handles the confirmation and the money, so there's no need to move the conversation elsewhere or make "side deals."

Prior studies

Research highlights that these transaction-heavy models are much more reliable. They reduce "user uncertainty" because the system structures every step of the interaction, making the whole market run more efficiently.

Strengths

- Clear-cut process: It removes the guesswork—you know exactly when a place is booked or paid for.
- Seamless experience: Everything is faster and easier for the user because it's all in one system.
- Paper trail: It keeps a formal record of every booking and payment, which is great for security.

Limitations

- Technical heavy lifting: Building and maintaining these systems is much more complex.
- Strict rules: The platform has to enforce policies (like cancellations), which require more active management.
- Less flexibility: It can be too "rigid" for people who prefer making informal, custom arrangements.

2.2.4 Profile-Based matching

Overview

This model focuses on personalization. Instead of making you scroll through every available home, the system tries to match you with the specific places that fit your lifestyle and needs.

How it works

It's a data-driven approach. You give the platform your details like your budget, must-have features, and preferred neighborhood. The system then filters through the database and suggests the best options for you, almost like a "recommendation engine."

Prior studies

Studies show that these matching systems make decision-making much faster. By focusing on user preferences, platforms can significantly boost user satisfaction and make the search process feel way more efficient.

Strengths

- Saves time: You don't waste hours looking at places you'd never actually rent.
- Highly relevant: The results are tailored specifically to what you asked for.

- Better experience: It makes the whole process feel personal and less overwhelming.

Limitations

- Data dependent: If the user doesn't provide enough detail (or the data is old), the matches won't be good.
- Hit or miss: No algorithm is perfect; sometimes it might miss a great option just because it didn't "check a box."
- "Black box" logic: It’s not always clear why the system chose one house over another, which can be frustrating for some users.

2.2.5 Integrated Communication

Overview

This model is all about keeping everything in one place. Instead of jumping between communication apps, all conversations between the tenant and the owner happen directly inside the platform.

How it works

Messaging, scheduling, and follow-ups are handled inside the system, often accompanied by notifications and logging to maintain a clear record of communication.

Prior studies

Studies indicate that integrating communication within accommodation platforms can improve coordination and reduce friction in the booking and management process.

Strengths

- Stay Organized: You don’t have to hunt through different apps to find a message; it’s all in one "thread."
- Better Accountability: It’s harder for things to get lost in various apps or for someone to claim they didn't see a message.
- Easier Support: If a dispute happens, the platform can look at the chat logs to help resolve the issue quickly.

Limitations

- User Habit: Some people still prefer the "old way" and might try to take the conversation offline.
- Privacy & Tech: The system needs to be very secure and user-friendly to make people actually want to use it.
- Development Cost: Building a reliable, real-time chat system adds a lot of technical work for the developers.

2.2.6Trust, Verification, and Safety Mechanisms

Overview

safety net for the platform. It’s a layer of security designed to make both renters and owners feel confident that they aren’t being scammed and that the people they are dealing with are legitimate.

How it works

Mechanisms include identity verification, review and rating systems, and platform moderation. These features are usually implemented alongside other approaches to reinforce reliability.

Prior studies

Prior research highlights the importance of trust layers in digital marketplaces, showing their role in reducing risk and enhancing user engagement.

Strengths

- Peace of Mind: Users feel much safer making a deal when they see "Verified" badges.
- Fraud Prevention: It makes it much harder for scammers to post fake listings.
- Transparency: Public feedback keeps everyone on their best behavior and makes the market more honest.

Limitations

- Active Effort: It only works if people actually leave reviews and the platform actively enforces its rules.
- Costly to Run: Manually checking IDs and monitoring content takes a lot of time and money.

2.2.7 University-Assisted and Institution-Supported Approaches

Overview

This is a hybrid approach. The university doesn't own the buildings, but they help students find a place to live. works as a guided search where the school acts as a bridge between students and private landlords.

How it works

The university usually runs a preferred portal or a list of approved rental apartments. They might set basic rules for landlords to join the list and provide a support office to help students understand their leases or avoid scams.

Prior studies

Studies describe this as a way to give students structured guidance. It’s especially helpful for moving students into the private market without leaving them completely on their own.

Strengths

- Peace of Mind: Students feel better knowing the university has "vetted" the options.
- Support System: It gives students a place to go if they have questions about off-campus living.
- Safe: It offers more variety than dorms but more safety than searching randomly on the web.

Limitations

- Limited Control: Since the school doesn’t own the property, they can’t always fix issues between a landlord and a tenant.
- Partial View: The list might not show every available apartment in the city, it just shows the apartments allocated near to it.
- Inconsistent: Some universities have great support, while others only provide a basic list of links.

2.2.8 Fully University-Managed Housing

Overview

In fully university-managed housing, the institution owns, allocates, and administers all accommodations for students.

How it works

It is a formal administrative process. Students apply, the school checks their eligibility, and rooms are assigned based on specific rules (like seniority or department). Everything is standardized and handled by the university's housing office.

Prior studies

Research has examined fully managed housing as a structured solution for student accommodation, highlighting its role in providing security and predictability while noting capacity limitations.

Strengths

- Maximum Security: These are usually the safest and most organized places for students to live.
- Zero Guesswork: Rules, costs, and expectations are clear from day one.
- Convenience: Students don't have to deal with external utility companies or private contracts.

Limitations

- Capacity Issues: Most schools don't have enough beds for every student, leading to long waitlists.
- Less Freedom: Dorm life has a lot of rules and "red tape" compared to living in a private apartment.
- One-Size-Fits: All: It lacks the flexibility or variety that some students might want.Listing-Based Marketplace.

2.3 Analysis of Existing Systems and Market Solutions

To evaluate current approaches to student accommodation, four representative systems have been selected. Each illustrates a different model of housing provision, reflecting the diversity of existing solutions. The analysis focuses on target users, core functionalities, system control, verification mechanisms, scalability, and limitations, in line with system design considerations.

2.3.1 MUST Student Housing Announcements (University-Announced Accommodation)

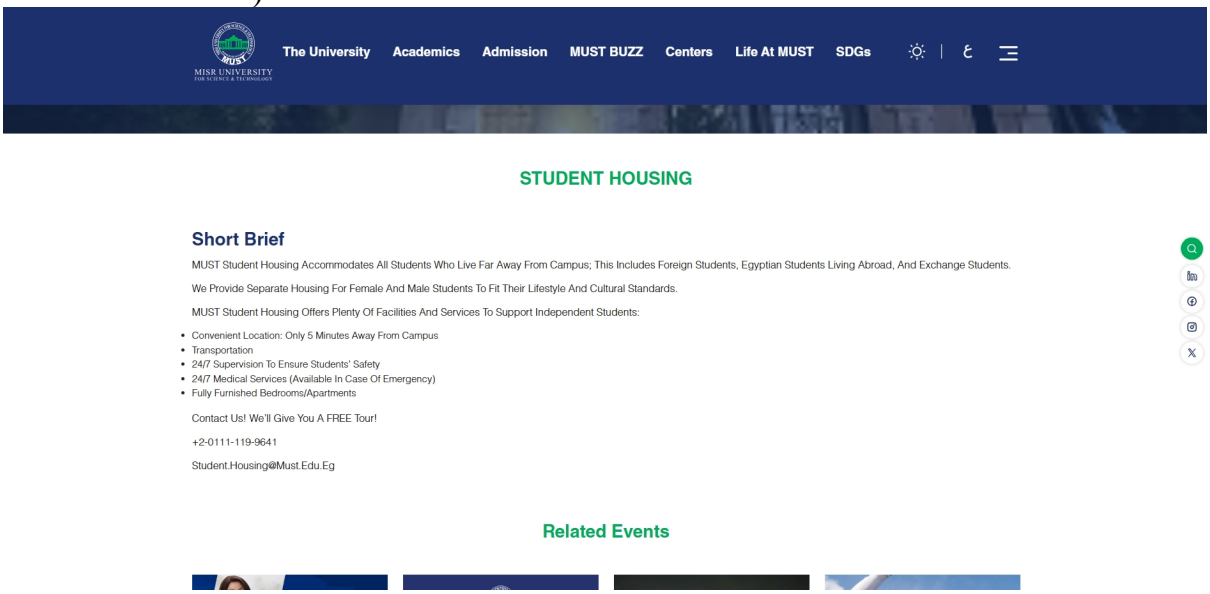

Figure 6:MUST Landing Page

Target Users

- Students of a specific university; only those eligible for university-provided housing.

Core Functionalities

- Display available rooms or dormitories
- Provide contact information for the housing office
- Basic descriptions of facilities and location

System Control

- Managed internally by the university, the website acts as an announcement platform only.
- No booking or payment features on the website.

Verification Mechanisms

- Verification is implicit via university administration; students must meet eligibility criteria.

Scalability & Flexibility

- Limited to students of a particular university.
- cannot scale to multiple institutions or external housing.

Strengths

- High trust and credibility
- Official and secure channel for students

Limitations

- Offline allocation and communication slow the process
- Limited housing availability
- No digital workflow or automation

2.3.2 University-Managed Housing Portal (Arizona State University)

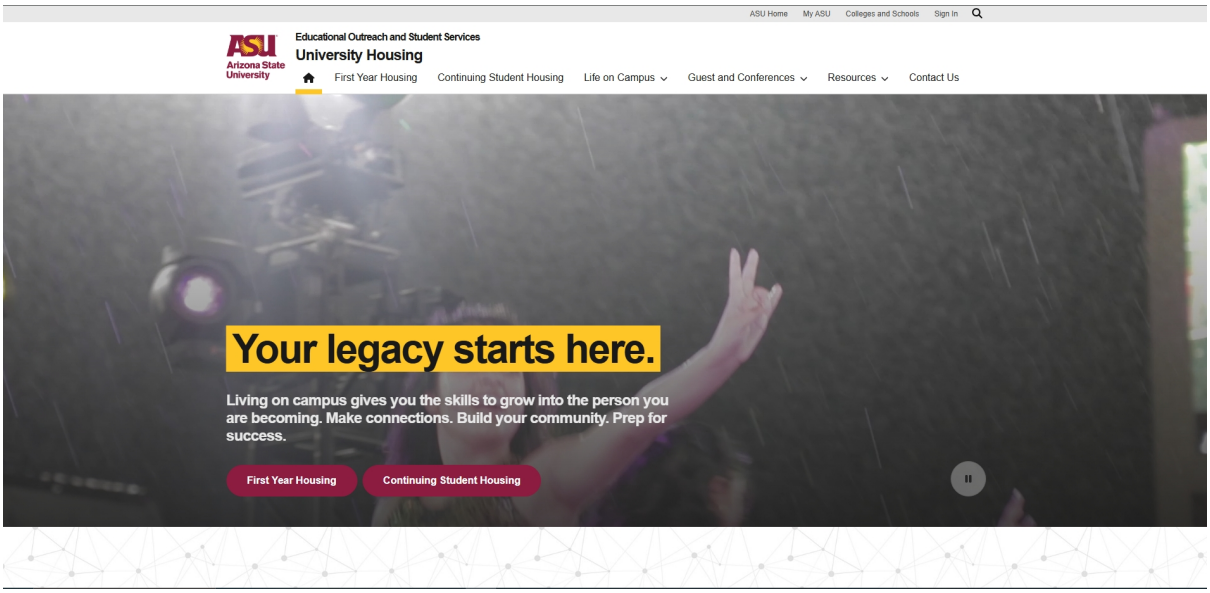

Figure 7:Arizona State University Landing page

Target Users

- Students enrolled at Arizona State University, eligible for university housing

Core Functionalities

- Submit housing applications online
- Select rooms or apartments according to preferences
- Track application status and confirm reservations
- Payment of deposits or fees through the portal
- Detailed information on facilities and services

System Control

- Full management of housing operations via the university portal
- Allocation, administration, and updates handled internally
- Not merely an announcement; a fully integrated administrative system

Verification Mechanisms

- Student identity verified through Single Sign-On (SSO)
- Ensures all applicants are enrolled students

Scalability & Flexibility

- Covers all university-managed housing facilities
- Flexible for different accommodation types (single rooms, shared apartments, halls)
- Not available for students outside the university

Strengths

- Secure and trustworthy
- Complete online workflow for application, booking, and administration
- Enables direct management of student housing

Limitations

- Restricted to enrolled students only
- Limited for students seeking external housing
- Expansion to other universities requires separate systems

2.3.3 Student.com (Student-Focused Private Platforms)

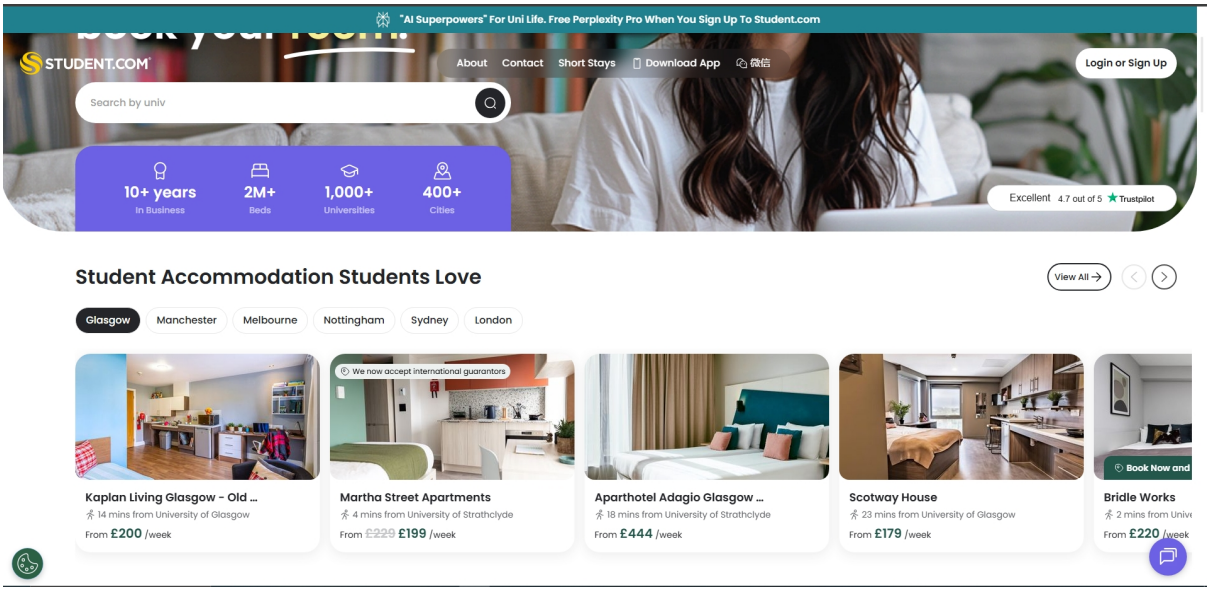

Figure 8:Student.com landing page

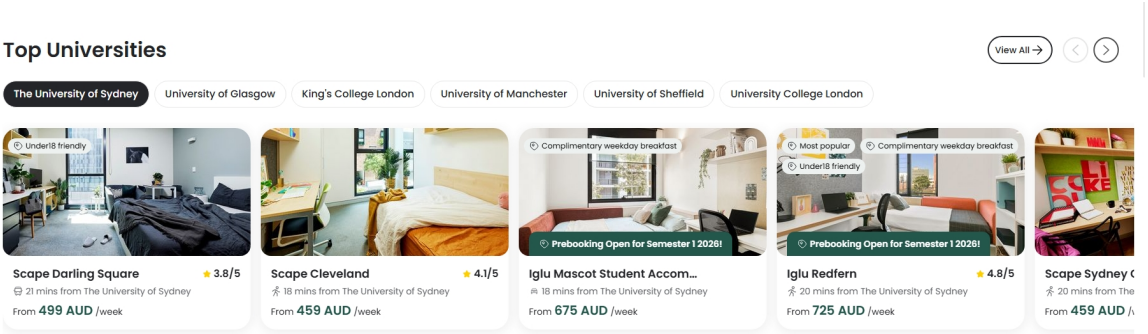

Figure 9:Top Universities Feature in Student.com

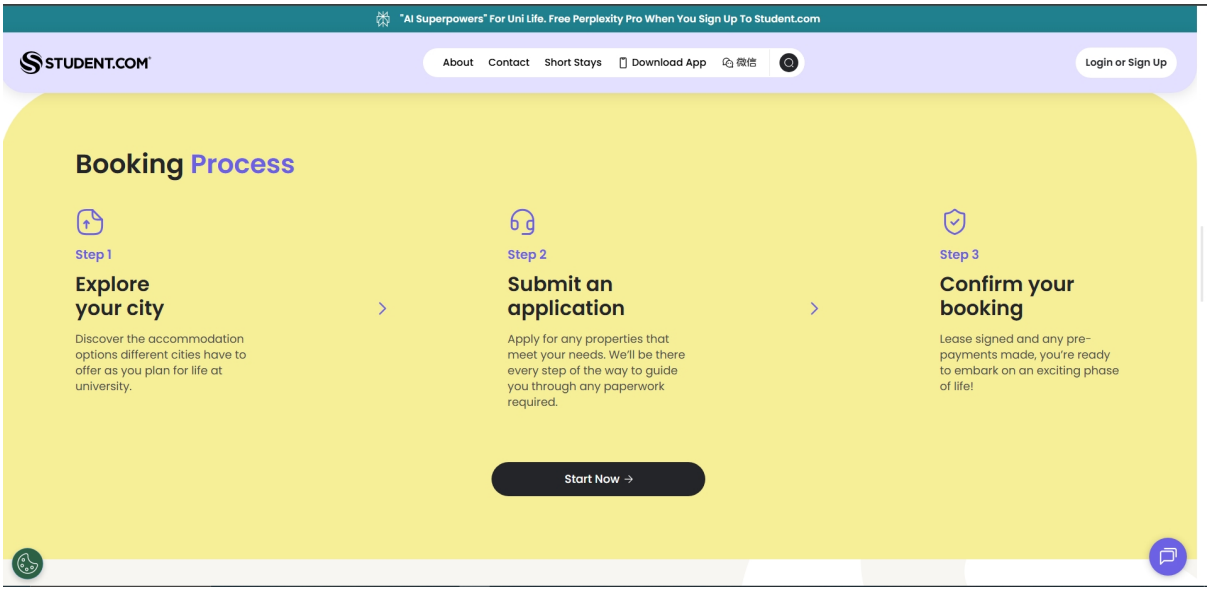

Figure 10:onboarding process of Student.com

Target Users

- Students seeking purpose-built accommodation near universities internationally.

Core Functionalities

- Search and filter by location, price, and type of student accommodation
- Online booking and payment for student halls and apartments
- Reviews and ratings for properties
- Student verification (registration required)

System Control

- Platform facilitates reservations and payments; landlords manage availability and rules.

Verification Mechanisms

- Student and property verification through the platform; standards enforced by Student.com policies.

Scalability & Flexibility

- Supports multiple countries and cities
- Variety of accommodation types, primarily purpose-built student halls

Strengths

- Strong focus on student-specific needs
- Reliable booking system
- Institutional-style listings with trust mechanisms

Limitations

- Mainly purpose-built student accommodation; less flexible for private rentals
- Higher fees compared to direct landlord communication
- University integration limited to information, not management

2.3.4 Airbnb (Booking-Centric General Platforms)

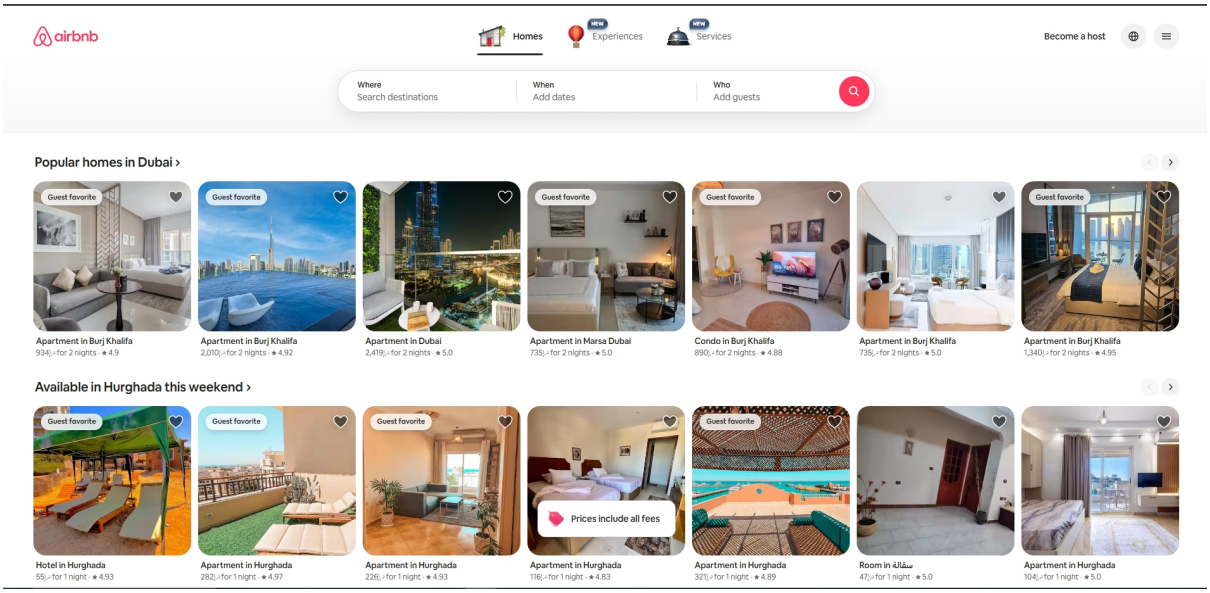

Figure 11:Airbnb landing page

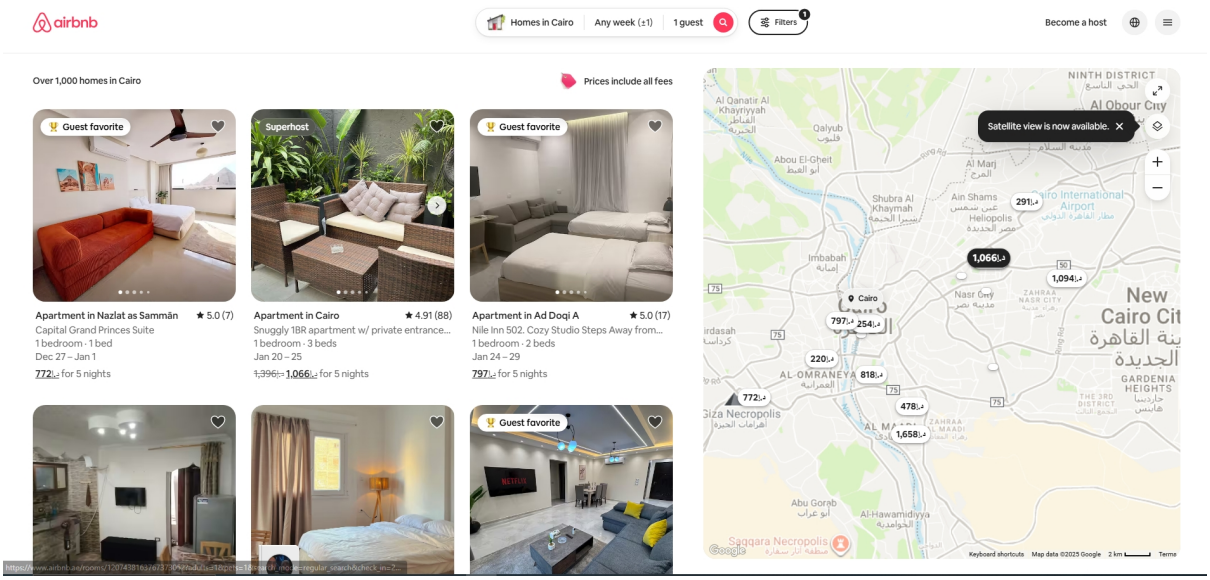

Figure 12:Airbnb browsing feature

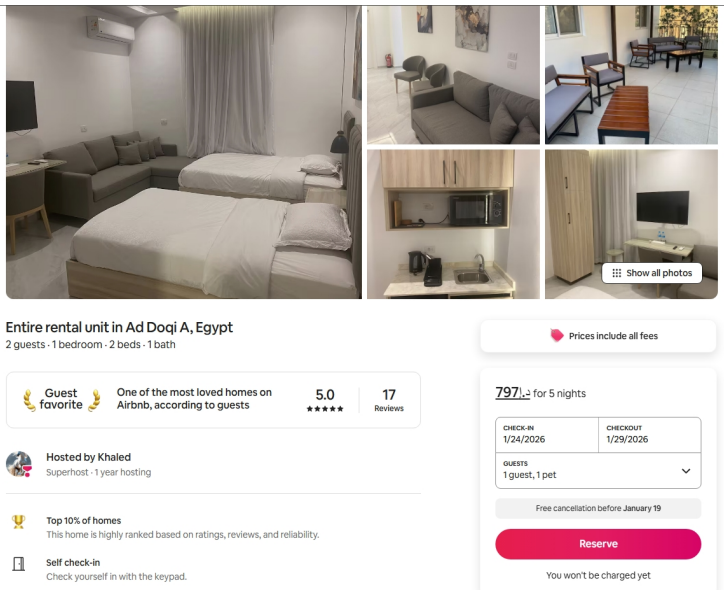

Figure 13:Example on a property in airbnb

Target Users

- General users seeking short- or medium-term accommodation, not student-specific

Core Functionalities

- Integrated booking and payment system
- Reviews and ratings
- Live availability management

System Control

- Full transaction management within the platform; minimal external communication needed.

Verification Mechanisms

- Basic verification of users and properties
- Trust through platform policies and reviews

Scalability & Flexibility

- Highly scalable; supports multiple locations and accommodation types
- Designed for short-term stays, limited alignment with academic calendars

Strengths

- Structured, reliable workflow
- Strong trust via reviews and policies
- Wide range of housing options

Limitations

- Not tailored for long-term student housing
- Lacks university integration
- Less flexibility for academic-specific arrangements

2.3.5 Facebook Groups, OLX, Dubizzle (Informal Listing-Based Platforms)

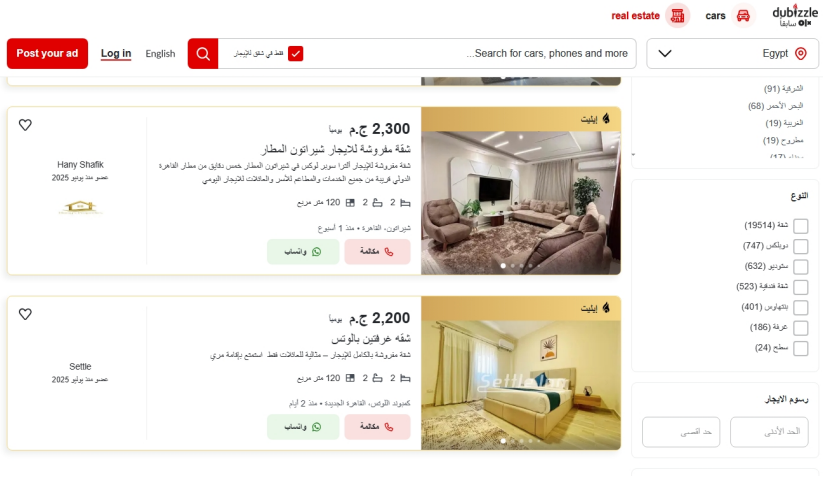

Figure 14:Dubbizle

Target Users

- Any individual looking for accommodation; not student-specific

Core Functionalities

- Post and browse housing listings
- Direct contact via messaging or phone
- Informal peer reviews (if any)

System Control

- Platform acts only as an intermediary for listings
- All communication and transactions occur off-platform

Verification Mechanisms

- Minimal or no verification; high risk of scams or misinformation

Scalability & Flexibility

- Very flexible and scalable; unlimited postings and locations

Strengths

- Free or low-cost
- Large variety of listings
- Easy to access

Limitations

- Low reliability and safety
- No structured booking or workflow
- Communication not tracked by platform

2.4 Conclusion: The Case for Saknny as a Comprehensive Solution

The preceding sections have established the foundational requirements for a robust student accommodation platform (Section 2.1) and reviewed the strengths and limitations of various existing industry approaches (Section 2.2). This conclusion synthesizes these findings to demonstrate why Saknny is uniquely positioned to address the complex challenges in the student housing market, effectively bridging the gaps left by existing models.

2.4.1 Synthesis of Foundational Requirements and Existing Gaps

Modern accommodation platforms must satisfy five core technical and operational foundations: Web-Based Architecture, Multi-Role User Systems, Scalability, Data Management, and Security, Trust, and Verification. A critical analysis of the eight related approaches reveals that each, while excelling in a specific area, fails to comprehensively integrate all five foundations, particularly in the context of the student-landlord-university ecosystem.

The following table illustrates how the limitations of the existing approaches directly translate into a failure to meet one or more of the core foundational bases defined in Section 2.1:

Related Approach (Section 2.2)	Primary Limitation (Gap)	Foundational Base Missed (Section 2.1)
1.Centralized Accommodations	Information accuracy and listing quality can vary, requiring constant quality assurance.	Security, Trust, and Verification (Lack of inherent verification mechanisms).
2.Listing-Based Marketplace	Lack of platform control, offline communication, and significant trust issues.	Multi-Role User Systems (No structured roles/controls beyond basic listing), Security, Trust, and Verification (Trust is the biggest hurdle).
3. Booking-Centric & Transaction-Oriented	High technical complexity, strict rules, and lack of flexibility for custom arrangements.	Scalability (High technical heavy-lifting and maintenance complexity can hinder horizontal growth).
4.Profile-Based Matching	Data dependency and "black box" logic can lead to missed opportunities and user frustration.	Data Management and System Consistency (Over-reliance on user-provided data, risking inconsistency).
5.Integrated Communication	User habit of moving conversations offline, high development cost for secure, real-time chat.	Security, Trust, and Verification (If users move off platform, the secure channel is bypassed).
6. Trust, Verification, and Safety Mechanisms	Costly to run, requires active effort from users (reviews), and is often an add-on, not a core feature.	Multi-Role User Systems (Verification is often applied to only two roles, missing the institutional oversight).
7.University-Assisted Approaches	Limited control over landlord-tenant issues, partial view of the market, and inconsistent support quality.	Web-Based Platform Architecture (Often a simple list or portal, not a comprehensive, multi-tiered application).
8. Fully University-Managed Housing	Severe capacity issues, lack of flexibility, and a "one-size-fits-all" approach.	Scalability (Capacity limitations inherently restrict horizontal scalability).

2.4.2 Saknny: The Integrated and Trust-Centric Solution

Saknny is designed as a comprehensive web application that strategically integrates the strengths of the most effective approaches while directly addressing the limitations identified above. Its core value proposition lies in its ability to simultaneously satisfy all five foundational bases of Section 2.1; a feat no single existing approach achieves.

1. Trust and Verification as a Core Feature (Addressing Gaps in 2.2.1, 2.2.2, 2.2.5): Unlike Centralized Accommodations (2.2.1) where trust is an afterthought, Saknny makes Security, Trust, and Verification (Section 2.1.5) a core capability. The requirement for Verified and Detailed Listings ensures information accuracy, mitigating the primary risk of the centralized model. Furthermore, the Secure Communication & Booking feature (integrated messaging and streamlined workflow) keeps the entire transaction on-platform, resolving the Offline Communication and Trust Issues inherent in the Listing-Based Marketplace (2.2.2) and the Integrated Communication approach (2.2.5).

2. Comprehensive Functionality (Addressing Gaps in 2.2.3, 2.2.4): Saknny adopts the best elements of the functional approaches:

- It incorporates the structure of the Booking-Centric model (2.2.3) with a streamlined booking workflow, ensuring a clear-cut process and paper trail.
- It leverages the personalization of the Profile-Based Matching model (2.2.4) through Unified Profiles for All Users, which allows for highly relevant search results and a better user experience.

In conclusion, Saknny is not a reinvention of a single existing model but a strategic convergence of the most successful elements, centralization, transaction management, personalization, and integrated trust—all unified under a robust, multi-role, and scalable Web-Based Platform Architecture. By incorporating the university as an active participant and making verification a prerequisite, Saknny provides the centralized, trustworthy, and efficient platform necessary to solve the problem of fragmented, time-consuming, and anxiety-inducing student accommodation search, making it the superior choice for the modern student housing ecosystem.

Chapter 3:

System Analysis and Design

3.1 Introduction

The purpose of this chapter is to translate the functional and non-functional requirements of the Saknny platform into a comprehensive technical blueprint. While the previous chapters (1 and 2) established the need for a centralized, trust-centric student housing solution, this chapter focuses on the system's "architecture" and "logic" the essential components that ensure the platform is secure, scalable, and efficient.

The design process follows a structured software engineering approach, moving from high-level process modeling to detailed data and interface design. This chapter is divided into the following key areas:

- System Requirements: A refinement of the core capabilities needed to serve students, landlords, and universities.
- Process Modeling: Using UML diagrams, such as Use Case and Activity diagrams, to visualize user interactions and the internal logic of features like the booking workflow.
- Data Design: Defining the relational database schema, ensuring data consistency and referential integrity across all system entities.
- Architectural Design: Detailing the multi-tier structure that separates the presentation layer from the business logic and centralized data management.
- Interface Design: Outlining the creation of responsive and dynamic user interfaces tailored to each stakeholder's needs.

By documenting these design decisions, this chapter serves as the definitive guide for the implementation phase, ensuring that the final product remains aligned with the project's primary aim: simplifying the relocation process for students through a transparent and verified platform.

3.2 System Requirements

This section defines the core functionalities and quality standards that Saknny must meet to fulfill its goal as a comprehensive web platform for student relocation. These requirements are derived from the project's aim to centralize the accommodation search process while ensuring transparency and safety for all stakeholders.

3.2.1 Functional Requirements

These requirements define the specific actions a University Administrator or Housing Office staff can perform within the portal:

- Student Enrollment Validation: The ability to verify whether a student requesting housing is currently enrolled in the institution.
- Verified Listings Oversight: Access to a filtered view of property listings that are geographically relevant to the university campus.
- Official Announcements: A tool to publish and manage housing-related news, deadlines, or safety warnings directly to the student dashboard.
- Analytics & Reporting: A dashboard to monitor metrics such as housing availability, average rental prices in the area, and general student satisfaction indicators.
- Stakeholder Support: A channel to provide guidance to students regarding their off-campus living rights and landlord-tenant relations.

3.2.2 Non-Functional Requirements

To maintain the integrity of the institution, this subsystem must adhere to strict quality standards:

- Security & RBAC: Access to the portal must be strictly limited to authorized university personnel using secure authentication.
- Data Accuracy: Reporting tools must provide real-time data from the centralized database to support data-driven decision-making.
- Auditability: The system should log administrative actions, such as when a student's status is validated or an announcement is posted.

3.2.3 Event Table for Sakknny

Consolidated System Event table

Table 3:Consolidated System Event Table

Module	Event / Trigger	Actor (Source)	Use Case (Verb)	System Response
1. Identity & Access	First-time open	Student	Select language	UI reloads in chosen language
1. Identity & Access	Authentication	Student/Admin	Authenticate user	Session created; role permissions loaded
2. Verification	Proof of Enrollment	Student	Upload document	Review task created for Admin
2. Verification	Fraud Signal Detected	System (Policy Engine)	Flag fraud risk	Risk flag added; Admin alerted; audit log written
2. Verification	Verification Queue	Admin	Review verification	Student flagged Eligible/Ineligible; notified
3. Catalog	Browse/Filter Housing	Student	Browse dorms	Returns list (optionally filtered by gender)
3. Catalog	Inventory Edit	Admin	Manage room inventory	Bed saved; availability recalculated
4. Application	Submit Request	Student	Submit application	Validated against gender/eligibility rules
4. Application	No Beds Available	System (Policy Engine)	Offer waitlist	Student prompted to join waitlist
4. Application	Final Approval	Admin	Approve application	Status = Approved; triggers allocation
5. Allocation	Bed Assignment	Admin	Assign bed	Record created; audit log written
5. Allocation	Inventory Change	System (Catalog)	Recalculate availability	Conflicts detected; Admins alerted if needed
6. Contract	Issue Lease	Admin	Issue lease	Lease generated; status = Pending

				Signature
6. Contract	Deadline Passes	System (Scheduler)	Expire lease	Lease expired; student/admin notified
6. Contract	Digital Signature	Student	Sign lease	Status updated to "LeaseSigned"
7. Payments	Pay Deposit/Rent	Student	Initiate payment	Gateway session returned; intent created
7. Payments	Refund Review	Admin	Approve/Reject refund	Processed via gateway; audit log written
8. Check-in	Key Issuance	Admin	Issue key	Key issued; student notified
9. Maintenance	Submit Ticket	Student	Create request	Ticket created; status = Open; student notified
9. Maintenance	SLA Time Breached	System (Scheduler)	Escalate ticket	Ticket escalated; supervisor alerted
10. Lifecycle	Room Change/Renewal	Student	Request change	Case created; status = Pending Review
10. Lifecycle	Inspection results	Admin	Record inspection	Damages captured; audit log written
11. Messaging	In-app/Email +1	System (Notification)	Notify applicant	Sent automatically on status change
11. Messaging	Post Announcement	Admin	Publish announcement	AnnouncementPublished; targeted students notified
12. Analytics	Dashboard View	Admin	View reports	Return occupancy, revenue, and SLA metrics
12. Audit	Sensitive Change	System	Write audit log	Stores who/what/when/before-after data
12. Surveys	Milestones	System (Scheduler)	Send survey	Survey request delivered to Student App

Admin Event Table

Table 4:Admin Event Table

Event	Trigger	Source	Use Case (Activity)	Destination	Response
Admin Authentication	Admin enters credentials	Admin Portal	Authenticate admin	Auth Service	Admin session created; role permissions loaded
Permission	Admin	Admin	Authorize	Auth Service /	Action allowed/denied;

Check	attempts restricted action	Portal	action	RBAC	denial logged
Verify Identity	Admin opens verification queue	Admin Portal	Review verification case	Profile Service	Case details displayed; decision options enabled
Document Approval	Admin approves a document	Admin Portal	Approve document	Profile Service	Document approved; case progress updated
Document Rejection	Admin rejects a document	Admin Portal	Reject document	Profile Service	Rejection reason recorded; student notified
Eligibility Approval	Admin marks student eligible	Admin Portal	Approve eligibility	Policy Engine	Student flagged Eligible (foreign/far-away satisfied)
Inventory Management	Admin adds/edits building or room	Admin Portal	Manage building/room inventory	Catalog Service	Record saved; availability recalculated; audit log written
Rule Configuration	Admin updates allocation constraints	Admin Portal	Configure allocation rules	Allocation Service	Rules updated (e.g., gender segregation); audit log written
Application Review	Admin starts review	Admin Portal	Start application review	Application Service	Status set to "Under Review"; audit log written
Application Finalization	Admin approves application	Admin Portal	Approve application	Application Service	Status = Approved; triggers allocation; student notified
Bed Assignment	Admin selects approved student	Admin Portal	Assign bed	Allocation Service	Assignment created OR blocked by constraints
Lease Issuance	Admin issues lease to student	Admin Portal	Issue lease	Contract Service	Lease generated; status = Pending Signature; student notified
Payment Monitoring	Admin opens payments dashboard	Admin Portal	Monitor payments	Billing Service	List of paid/unpaid/overdue balances returned
Maintenance Assignment	Admin assigns technician	Admin Portal	Assign maintenance ticket	Maintenance Service	Ticket assigned; schedule updated; audit log written
Incident	Admin	Admin	Review	Maintenance	Actions/notes recorded;

Review	reviews incident report	Portal	incident report	Service	student optionally notified
Official Announcement	Admin posts announcement	Admin Portal	Publish announcement	Communications Service	Announcement published; targeted students notified
Analytics Reporting	Admin opens analytics dashboard	Admin Portal	View reports	Reporting Service	Metrics returned (occupancy, demand, revenue, etc.)
Audit Logging	Any sensitive change occurs	Any Subsystem	Write audit log	Audit Service	Audit entry stored (who/what/when/before-after)

Student Event Table

Table 5:Student Event Table

Event	Trigger	Source	Use Case (Activity)	Destination	Response
Registration	Student taps "Sign Up"	Student App	Register account	Auth Service	Account created; verification required
Authentication	Student taps "Login"	Student App	Authenticate user	Auth Service	Session created or error returned
Profile Setup	Student edits personal info	Student App	Update profile	Profile Service	Profile updated; audit log written
Verification Submission	Student uploads ID/Proof	Student App	Request verification	Profile Service	Case opened; status set to "Pending"
Catalog Search	Student opens catalog	Student App	Browse/Filter dorms	Catalog Service	Filtered dorm list returned (per gender rules)
Application Submission	Student taps "Submit Application"	Student App	Submit application	Application Service	Validated (docs/eligibility); status = Submitted
Waitlist Entry	No beds available	System/Policy Engine	Join waitlist	Application Service	Waitlist entry created; student notified
Status Tracking	Student opens status page	Student App	Track application status	Application Service	Current status + next actions returned
Lease Signing	Student signs digital lease	Student App	Sign lease	Contract Service	Lease signed; status updated; admin notified

Payment Initiation	Student taps "Pay Deposit/Rent"	Student App	Initiate payment	Billing Service	Gateway session returned; intent created
Payment Confirmation	Gateway confirms success	Payment Gateway	Confirm payment	Billing Service	Balance updated; receipt/invoice available
Check-in Execution	Students arrive & taps check-in	Student App	Confirm check-in	Check-in Service	Check-in record created; instructions displayed
Maintenance Request	Student submits ticket	Student App	Create maintenance request	Maintenance Service	Ticket created; status = Open; student notified
Room Change Request	Student requests change	Student App	Request room change	Residency Service	Case created; status = Pending Review
Checkout Initiation	Student starts checkout	Student App	Initiate checkout	Residency Service	Checkout record created; instructions shown
In-App Messaging	Student sends message	Student App	Send message	Comms Service	Conversation updated; admin notified

3.3 Process Modelling

3.3.1 Use Case Diagram

3.3.2 Data Flow Diagram (DFD)

3.3.3 Activity Diagrams

3.4 System Architecture

The architecture of Saknny is designed to support a robust, scalable, and secure environment for student relocation. It follows a structured approach to ensure that the presentation, logic, and data management layers remain distinct yet integrated.

3.4.1 Multi-Tier Architectural Structure

The system adopts a Multi-Tier Architecture, which logically separates different functional components to improve maintainability and allow for independent updates to each layer.

- Presentation Tier (User Interface): This is the top-most level of the application, consisting of the dynamic web interfaces that students and university administrators interact with. It focuses on responsiveness and advanced filtering for property searches.
- Business Logic Tier (Application Core): This tier handles the core application logic, including authentication, authorization, property management, and booking workflows. It acts as the intermediary between the user interface and the database.
- Data Management Tier (Database): This layer consists of the relational database responsible for storing structured information such as user profiles, property listings, and transactional records while ensuring referential integrity.

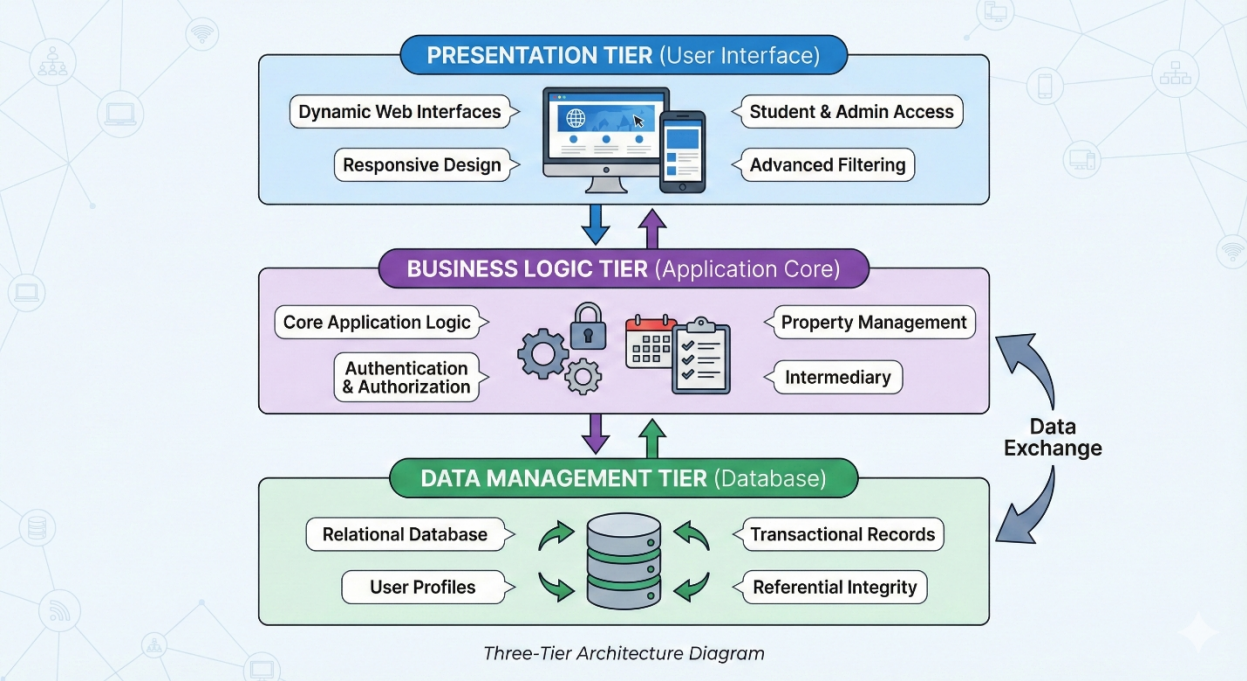

Figure 15:Architecture Diagram

3.4.2 MVC Design Pattern

To further organize the internal logic of the system, Saknny utilizes the Model-View-Controller (MVC) design pattern. This pattern is essential for separating the internal representations of information from the ways that information is presented to the user.

- Model: Represents the data structure and business rules. It manages the student profiles, housing catalog, and lease records.
- View: The visual representation of the data (the UI). It renders the dashboards, property pages, and analytics charts based on the information provided by the Model.
- Controller: Processes user input (like a booking request or an enrollment validation) and coordinates between the Model and the View to update the system state.

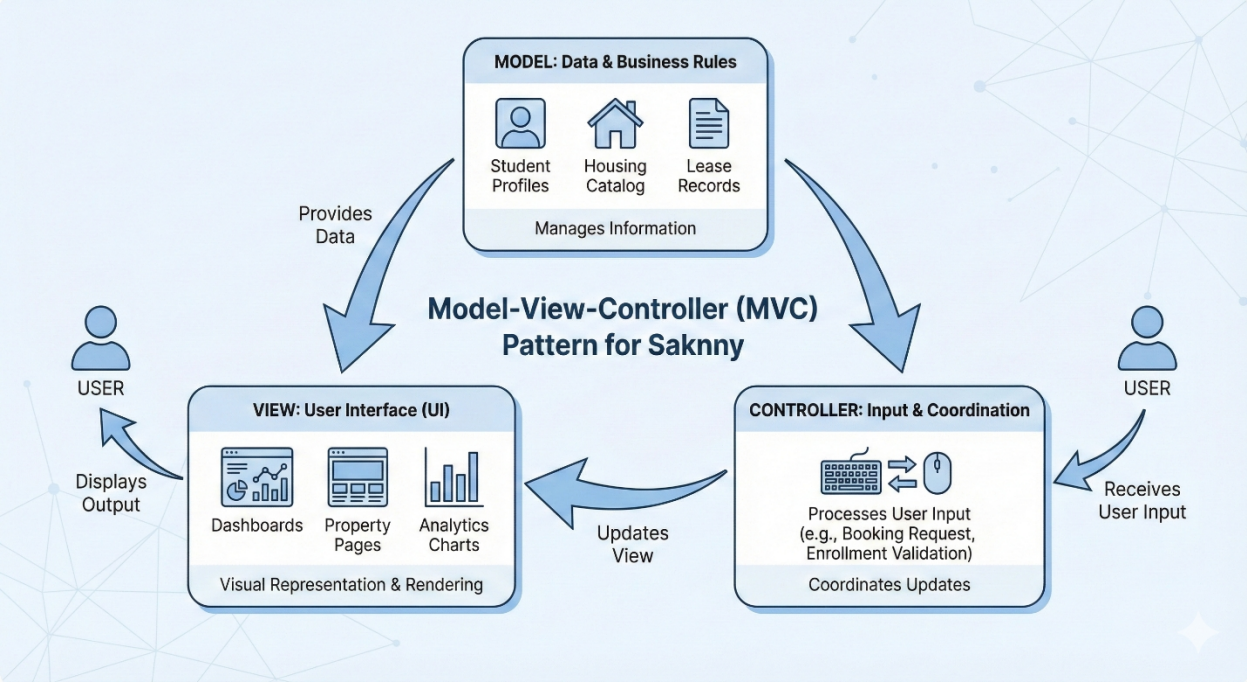

Figure 16:MVC Model of Saknny

3.4.3 Scalability and Distribution

The system is built with both Vertical and Horizontal Scalability in mind.

- Vertical Scalability: Involves increasing resources (such as CPU or RAM) to support growth in user activity within a single university.
- Horizontal Scalability: Enables the platform to support additional institutions and geographical regions by adding more servers to distribute the workload without requiring fundamental architectural changes.

3.4.4 Role-Based Access Control (RBAC)

Security is maintained through a strict Role-Based Access Control foundation. This ensures that each user (Student, Landlord, or University Admin) can only access the assets and rules relevant to their specific role. For instance, only authorized university personnel can access the enrollment validation and administrative analytics tools

3.5 Data Design

The data design for the Saknny University Portal focuses on establishing a robust relational structure that ensures data integrity, minimizes redundancy through normalization, and supports complex institutional workflows like room allocation and meal plan management. This section translates the Entity-Relationship Diagram (ERD) into a formal technical specification.

3.5.1 Entity Relationship Diagram (ERD)

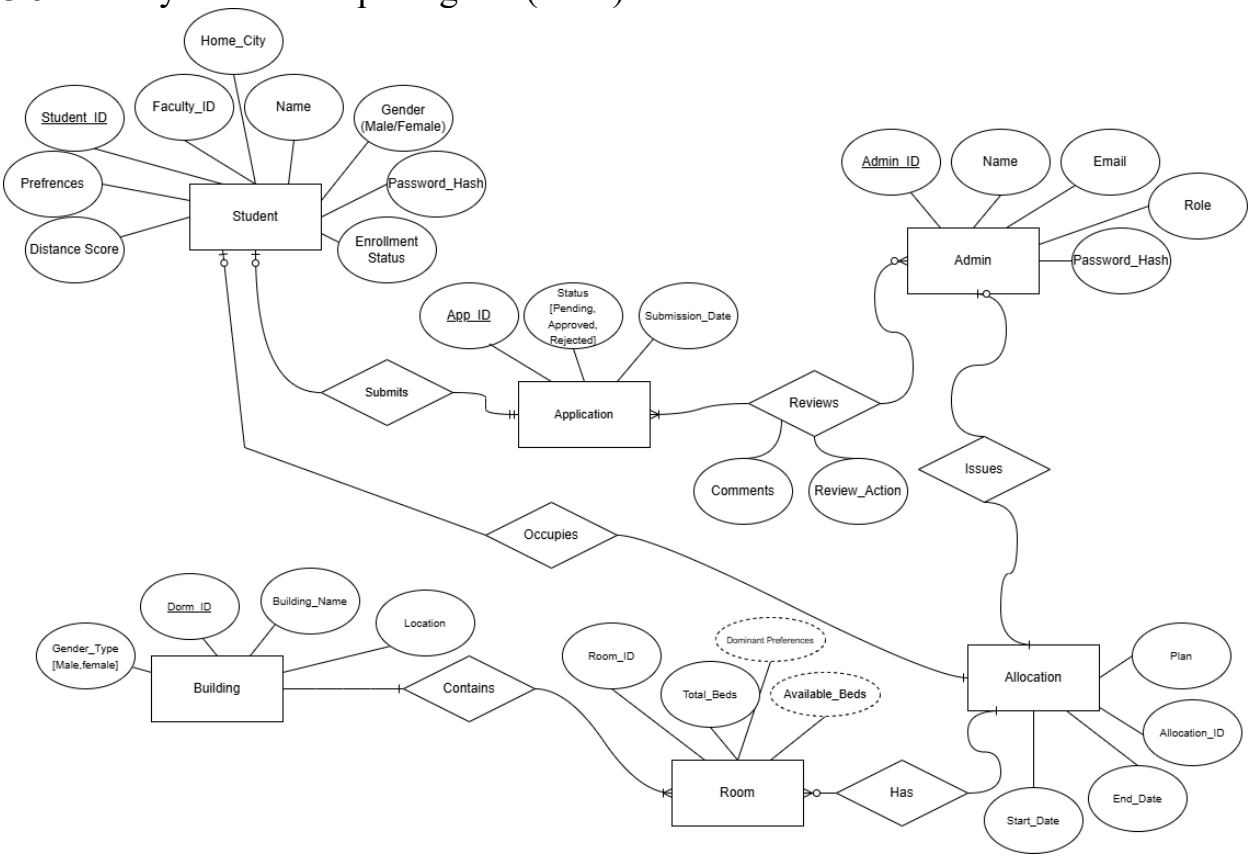

Figure 17:Entity Relationship Diagram

Table 6:Entity Relationship breakdown

Relationship	**Cardinality**	**Modality (Participation)**	**Logic**
Student ↔ Application	1:1	Optional → Mandatory	A student may apply; an application must belong to a student.
Admin ↔ Application	M: N	Optional → Mandatory	Multiple admins can review one app; apps must be reviewed.
Building ↔ Room	1: N	Mandatory → Mandatory	A building must have rooms; a room must be in a building.
Room ↔ Allocation	1: N	Optional → Mandatory	A room can be empty; an allocation must have a

			room.
Student ↔ Allocation	1:1	Optional → Mandatory	A student gets an allocation only if approved.
Admin ↔ Allocation	1: N	Optional → Mandatory	Admin issues the final assignment for accountability.

3.5.2 Data Integrity Constraints

Referential Integrity

An Allocation cannot exist without a verified Student and an available Room.

Gender Rule

Students.Gender must match Dormitories.Gender_Type.

Capacity Logic

Rooms.Available_Beds must be greater than zero for an Allocation to be valid.

3.5.3 Mapping

Table: Application_Reviews				
Review_ID (PK)	App_ID (FK referencing Applications)	Admin_ID (FK referencing Admins)	Review_Action (e.g., 'Document Verified', 'Eligibility Checked')	Comments

Table: Admin				
Admin_ID (PK)	Name	Email	Password_Hash	Role

Table: Students								
Student_ID (PK)	Faculty_ID (Unique)	Password_ Hash	Name	Gender	Home_City	Enrollment_Status	Distance_Score	Prefrences

Table: Applications				
App_ID (PK)	Student_ID (FK referencing Students)	Preferred_Dorm_ID (FK referencing Buildings)	Status (Pending, Approved, Rejected)	Submission_Date

Table: Buildings			
Dorm_ID (PK)	Building_Name	Gender_Type (M/F)	Status (Active/Maintenance)

Table: Rooms					
Room_ID (PK)	Dorm_ID (FK referencing Buildings)	Room_Number	Total_Beds	Available_Beds (Derived/Calculated)	Dominant Preferences

Table: Allocations				
Allocation_ID (PK)	Student_ID (FK referencing Students)	Room_ID (FK referencing Rooms)	Admin_ID (FK referencing Admins)	Plan (Breakfast/Full Board)

3.5.4 Data Dictionary (Physical Schema)

Table: Admin

Attribute	Data Type	Description	Constraints
Admin_ID	INT	Unique internal identifier for the admin[3].	**Primary Key**
Name	VARCHAR(100)	The full name of the university staff member[4].	NOT NULL
Email	VARCHAR(100)	Institutional email used for login[5].	Unique, NOT NULL
Password_Hash	VARCHAR(255)	Securely hashed password for authentication[6].	NOT NULL
Role	VARCHAR(50)	Defines permissions (e.g., Housing Manager, Staff)[7].	NOT NULL

Table: Students

Attribute	Data Type	Description	Constraints
Student_ID	INT	Unique identifier for the student.	**Primary Key**
Faculty_ID	VARCHAR(20)	Official university faculty ID number.	Unique, NOT NULL
Name	VARCHAR(100)	The full name of the student.	NOT NULL
Gender	ENUM('M','F')	Required for **gender segregation** rules.	NOT NULL
Home_City	VARCHAR(50)	Used to determine if a student lives "far-away".	NOT NULL
Enroll_Status	BOOLEAN	Flag set by Admins to confirm university enrollment.	Default: FALSE
Distance_Score	DECIMAL(5,2)	Priority score calculated based on the student's home city.	Default: 0
Prefrences	VARCHAR(200)	Preferences of the student to be used for the matching agent	Nullable

Table: Applications

Attribute	Data Type	Description	Constraints
App_ID	INT	Unique identifier for the housing application.	**Primary Key**
Student_ID	INT	Links to the student submitting the request.	**Foreign Key**
Dorm_Pref_ID	INT	Links to the building chosen by the student.	**Foreign Key**
Status	VARCHAR(20)	State of the request: Pending, Approved, or Rejected.	Default: 'Pending'
Submit_Date	TIMESTAMP	The exact date and time of application submission.	NOT NULL

Table: Application_Reviews (M:N Link Table)

Attribute	Data Type	Description	Constraints
Review_ID	INT	Unique identifier for the specific review action.	**Primary Key**
App_ID	INT	The application being reviewed.	**Foreign Key**

Admin_ID	INT	The administrator performing the review.	**Foreign Key**
Review_Action	VARCHAR(50)	The specific step taken (e.g., 'ID Verified').	NOT NULL
Review_Time	TIMESTAMP	When the review action occurred.	NOT NULL
Comments	TEXT	Optional feedback or rejection reasons.	NULLABLE

Table: Building

Attribute	**Data Type**	**Description**	**Constraints**
Dorm_ID	INT	Unique identifier for the building.	**Primary Key**
Building_Name	VARCHAR(100)	Common name of the dormitory building.	NOT NULL
Gender_Type	ENUM('M','F')	Restricts the building to a specific gender.	NOT NULL
Status	VARCHAR(20)	Indicates if the building is Active or under Maintenance.	Default: 'Active'

Table: Room

Attribute	**Data Type**	**Description**	**Constraints**
Room_ID	INT	Unique identifier for the room.	**Primary Key**
Dorm_ID	INT	Links the room to its parent building.	**Foreign Key**
Room_Number	VARCHAR(10)	The room label (e.g., 'A-101').	NOT NULL
Total_Beds	INT	The maximum number of students allowed.	NOT NULL
Available_Beds	INT	**Derived attribute**; current vacancy count.	Calculated
Dominant_Preferences	VARCHAR(50)	**Derived attribute**; the dominant preference from the student in the same room	Computed

Table: Allocation

Attribute	Data Type	Description	Constraints
Allocation_ID	INT	Unique identifier for the bed assignment.	**Primary Key**
Student_ID	INT	The student assigned to the bed.	**Foreign Key**
Room_ID	INT	The room where the student is staying.	**Foreign Key**
Admin_ID	INT	The admin who authorized this specific assignment.	**Foreign Key**
Plan	VARCHAR(30)	Dining plan: Breakfast Only or Full Board.	NOT NULL

References

[1] Misr University for Science and Technology, "MUST Student Housing," must.edu.eg. [Online]. Available: https://must.edu.eg. [Accessed: Jan. 13, 2026].

[2] Arizona State University, "University Housing," housing.asu.edu. [Online]. Available: https://housing.asu.edu. [Accessed: Jan. 13, 2026].

[3] Student.com, "Student Accommodation Students Love," student.com. [Online]. Available: https://www.student.com. [Accessed: Jan. 13, 2026].

[4] Airbnb, "Vacation Rentals, Homes, Experiences & Places," airbnb.com. [Online]. Available: https://www.airbnb.com. [Accessed: Jan. 13, 2026].

[5] Dubizzle Egypt, "Properties for Rent in Egypt," dubizzle.com.eg. [Online]. Available: https://www.dubizzle.com.eg. [Accessed: Jan. 13, 2026]